

2026년도 전국기능경기대회

직 종 명	클라우드컴퓨팅	과 제 명	Small Challenge	과제번호	제 2과제
경기시간	4시간	비 번 호		심사위원 확 인	(인)

1. 요구사항

- 1) NoSQL
- 2) CDN Function
- 3) EKS Scaling
- 4) EKS 011y

2. 선수 유의사항

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요 시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방합니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 문제에 제시된 괄호박스 <> 는 변수를 뜻하므로 선수가 적절히 변경하여 사용해야 합니다.
- 6) 문제 풀이와 채점의 효율을 위해 보안 그룹의 80/443 Outbound는 Anyopen 하도록 합니다.
- 7) 모든 리소스는 과제지에 명시된 리전에 구성하며, 문제에서 주어지지 않는 값들은 AWS Well-Architected Framework 6 pillars를 기준으로 적절한 값을 설정해야 합니다.
- 8) 모든 이름, 태그, 변수는 대소문자를 구분하며, Tag 값을 올바르게 설정하도록 유의합니다.
- 9) 과제 종료 전 실행 중인 테스트 및 부하를 중지하여 서버에 문제가 없도록 해야 합니다.
- 10) 문제지에 표기된 중괄호 { }는 동일 패턴의 리소스를 축약 표기한 것이며, 선수가 중괄호 내 값을 각각 전개하여 사용해야 합니다. (예: unicorn-subnet-pub-{a,b,c} -> unicorn-subnet-pub-a, unicorn-subnet-pub-b, unicorn-subnet-pub-c)
- 11) 모든 IAM Policy 및 리소스 기반 Policy(Key Policy, Bucket Policy 등)은 Principal: "*" 또는 Action: "*"과 같이 권한을 광범위하게 개방하여 작성해서는 안 되며, 각 리소스가 요구하는 최소 권한 원칙에 따라 권한을 부여해야 합니다.
- 12) 모든 EC2 인스턴스는 별도 명시가 없는 한 t3.small 사이즈와 Amazon Linux 2023 AMI를 사용합니다.

1) NoSQL (ap-southeast-1)

개요

당신은 BigBae Trains의 DevOps 엔지니어로, DynamoDB, Lambda, EC2를 사용하여 기차 티켓 예매 시스템을 설계하고 구축해야 합니다. 아래 지시에 맞게 구성하도록 합니다.

1. NoSQL

DynamoDB를 활용하여 좌석 단위의 동시 예약 제어와 이벤트 후처리가 가능한 시스템을 구성합니다. 예약된 좌석을 사용자별로 조회할 수 있도록 Global Secondary Index를 구성하며, 예약되지 않은 좌석은 해당 인덱스에 포함되지 않도록 합니다.

DynamoDB Streams를 활성화하여 예약 이벤트가 별도의 감사 테이블에 자동으로 적재되도록 합니다. 데이터 손실에 대비하여 PITR을 활성화 합니다. Pay per Request 모드를 사용합니다.

- Reservation Table & GSI

Reservation Table Name	bigbae-nosql-reservation-table
Partition/Sort Key	train_id (String), seat_id (String)
Stream	NEW_AND_OLD_IMAGES

GSI Name	gsi-user-reservations
Partition/Sort Key	user_id (String), reserved_at (String)
Projection	ALL

- Audit Table

Reservation Table Name	bigbae-nosql-audit-table
Partition Key	event_id (String)

2. Conditional Write 구성

동일한 좌석에 대해 동시에 예매 요청이 발생하더라도 가장 먼저 선점한 단 한 건만 성공해야 합니다. Python 애플리케이션은 좌석 예약 시 status = "available" 조건을 만족하는 경우에만 좌석 상태를 "reserved"로 변경하고 version 값을 1 증가시킵니다. 조건이 충족되지 않을 경우 409 Status Code 와 함께 오류 메시지를 반환해야 합니다. GSI에는 예약된 좌석만 포함되어야 하며, 예약 취소 시 GSI Key 속성을 제거하여 인덱스에서 자동 제외되도록 합니다.

3. Streams 처리 구성

예약 또는 취소 이벤트가 발생할 때마다 Python3.13 기반의 Lambda 함수가 DynamoDB Streams로부터 이벤트를 수신하여 Audit 테이블에 기록을 30초 내로 적재하도록 합니다. 적재되는 항목은 기차 정보, 좌석 정보, 사용자 정보, 발생 시각을 포함합니다. 지급된 lambda.py를 사용하여 함수를 구성하며, 코드는 수정하지 않습니다. Lambda 함수는 아래와 같이 구성합니다.

Function Name	bigbae-nosql-reservation-audit
Timeout	30 seconds
Trigger	DynamoDB Streams (bigbae-nosql-reservation-table)

4. Application 배포

Python Flask로 작성된 App app.py를 bigbae-nosql-app-ec2 이름을 가진 EC2 인스턴스에 배포

합니다. TCP 8080 포트로 바인딩되며, DynamoDB 테이블과 GSI를 사용하여 좌석 예약, 취소, 조회 기능을 제공합니다. 외부에서 Public IP를 통해 요청을 보낼 수 있어야 합니다.

- Environment Variable & API Spec

AWS_REGION	ap-southeast-1
TABLE_NAME	bigbae-nosql-reservation-table
GSI_NAME	gsi-user-reservations

Path	Method	Request body	Status Code : Response Example
/healthcheck	GET	X	200 OK
/reserve	POST	{"train_id":"<id>","seat_id":"<id>","user_id":"<id>"}	200 : {"status":"reserved","seat_id":"<id>","version":"<n>"}, 409 : {"error":"already reserved"}
/cancel	POST	{"train_id":"<id>","seat_id":"<id>","user_id":"<id>"}	200 : {"status":"cancelled","seat_id":"<id>"}, 409 : {"error":"not owner"}
/seats/<train_id>	GET	X	200 : [{"seat_id":"<id>","status":"<status>","user_id":"<id or null>"}]
/my-bookings/<user_id>	GET	X	200 : [{"train_id":"<id>","seat_id":"<id>","reserved_at":"<ts>"}]

2) CDN Function (us-east-1)

개요

CloudFront, CloudFront Functions, KeyValueCollection, S3를 사용하여 엣지 기반의 Dynamic A/B 테스트 시스템을 구성합니다. 자세한 사항은 아래를 참고하여 구성합니다.

1. CDN 구성

SkillsPhone Inc.는 랜딩 페이지의 신규 디자인의 고객 반응을 살피기 위해, 사용자 A/B 테스트 시스템을 구성하고자 합니다. 접속 시 설정된 비율에 따라 A 또는 B 버전을 무작위로 할당 받으며, 할당받은 버전은 재접속 시에도 동일하게 유지되어야 합니다. A/B 노출 비율을 조정할 경우, 코드 재배포 없이 즉시 반영되어야 합니다. 정적 콘텐츠는 S3 버킷에 호스팅 하며, CloudFront Distribution만을 통해서 접근이 가능하도록 구성합니다. 모든 Public Access는 차단하며 OAC를 사용하여 보호하도록 합니다.

- S3 Bucket Name : skillspone-landing-ab-<ACCOUNT_ID 12자리>
- index_a.html 파일 업로드 위치 : /version-a/index.html
- index_b.html 파일 업로드 위치 : /version-b/index.html

2. KeyValueCollection 구성

A/B 노출 비율과 각 버전 별 경로를 CloudFront Function이 참조할 수 있도록 KeyValueCollection을 생성하여 아래의 세 개의 키 값을 정확히 보유하도록 합니다.

- KeyValueCollection Name : skillspone-cdn-ab-config

weight	0.3
version_a	/version-a/index.html
version_b	/version-b/index.html

3. Cloudfront Function 구성

CloudFront의 viewer-request 단계와 viewer-response 단계에 각각 연결되는 두 개의 함수를 작성합니다. 두 함수 모두 cloudfront-js-2.0 런타임을 사용하며 LIVE 스테이지로 발행합니다. viewer-request 함수는 위에서 생성한 KeyValueStore와 연결되도록 구성합니다.

- Viewer Request Function Name : skillssphone-cdn-ab-req-fn
- Viewer Response Function Name : skillssphone-cdn-ab-res-fn

viewer-request function 작동 방식

- Request Cookie에 x-sp-ab가 존재하는 경우, 해당 Cookie 값(a or b)에 따라 KeyValueStore의 version_a or b 값을 사용하여 Request URI를 재작성합니다.
- Request Cookie에 x-sp-ab가 존재하지 않는 경우, KeyValueStore의 weight 값을 읽어 0과 1 사이의 무작위 값이 weight 미만이면 b, 그렇지 않으면 a를 할당합니다.
- 할당된 version 값을 사용하여 Request URI를 재작성하고, 다음 단계에 전달하기 위해 요청 헤더 x-sp-ab-assigned의 값을 할당된 버전(a or b)으로 설정합니다.

viewer-response function 작동 방식

- Request header x-sp-ab-assigned 가 존재하는 경우에만 Response Header Set-Cookie에 x-sp-ab=<할당된 버전>; Path=/; Max-Age=86400을 추가하여 응답을 반환합니다.

4. Policy 구성

A/B 버전이 서로 다른 캐시 항목으로 보관 되도록 하기 위해 캐시 정책에 x-sp-ab 쿠키를 캐시 키에 포함하도록 합니다. Response Header 정책으로 Security Header를 추가하여 응답을 반환합니다. AWS Managed Policy는 사용하지 않도록 합니다.

- Cache Policy Name : skillssphone-cdn-ab-cache-policy
- Min/Default/Max TTL : 0 / 300 / 3600
- Cookies : whitelist (x-sp-ab)

5. CloudFront Distribution 구성

앞서 생성한 리소스를 고객에게 제공하기 위해 Pay-as-you-go 타입의 CloudFront Distribution을 생성합니다. HTTP로 접속 시 HTTPS로 리디렉션되어야 하며, viewer-request / response에서 앞서 생성한 함수와 캐시 정책을 연결하도록 합니다.

- Distribution Name : skillssphone-cdn-ab-distribution

3) EKS Scaling (ap-northeast-2)

개요

당신은 SkillsMarket, LLC.의 엔지니어로써 확장성 높고 유연한 EKS 아키텍처를 설계하여야 합니다. 블랙 프라이데이 등 행사 진행 시 트래픽 스파이크가 발생할 수 있으므로 빠르고 유연하게 Scale-out/in 되어야 합니다. 회사는 주문을 처리하기 위해 SQS를 사용하고, Queue 기반으로 스케일링 하기 위해 KEDA를 사용하며, 노드 스케일링을 위해 Karpenter를 사용합니다.

1. SQS

주문을 처리하기 위한 SQS Standard Queue를 생성합니다. KEDA가 해당 Queue의 메시지 수를 기반으로 Pod를 스케일링합니다. 명시되지 않은 값은 기본값을 사용합니다.

- Queue Name : skm-order-queue

2. EKS Cluster

서비스를 배포하기 위한 EKS 클러스터를 구축하고, 아래와 같이 Addon 설치를 위한 Managed NodeGroup을 구성합니다. VPC의 경우 선수가 자유롭게 구성하거나, 기본 VPC를 사용할 수 있습니다. taint를 통해 Addon NodeGroup에서 App이 실행되지 않도록 합니다.

- EKS Cluster Name / Version : skm-eks-cluster / 1.35
- Addon NodeGroup Spec

NodeGroup Name	skm-cluster-addon-ng
Node Tag	skm-cluster-addon-ng-node
Instance Size	t3.medium
Desired/Min/Max Size	1 / 1 / 1

3. Application

Python으로 작성된 app.py를 컨테이너 이미지로 빌드하여 skillsmkt Namespace에 배포합니다. App은 SQS Queue로부터 메시지를 수신하여 처리하는 Consumer로 동작하며, 적절히 SQS 메시지의 수신, 삭제 권한을 부여하도록 합니다. Healthcheck를 위한 API 엔드포인트가 존재하며, 8080 포트에서 동작합니다. App은 후술할 skm-app-nodepool에서만 구동되어야 합니다.

- Deployment Name / Initial Replicas : order-processor / 1
- Pod Resource Requests : CPU 500m, Memory 512Mi
- Environment Variables & API Spec

AWS_REGION	ap-northeast-2
SQS_QUEUE_URL	위에서 구성한 Queue URL
PROCESSING_TIME	3

Path	Method	Response
/healthz	GET	HTTP 200 {"status": "ok"}
/status	GET	HTTP 200 {"processed": <n>, "queue_url": "<url>"}

4. Pod Scaling

KEDA를 EKS Cluster에 설치하여 SQS Queue의 메시지 수에 따라 Pod 수가 자동으로 Scaling 되도록 구성합니다. KEDA Operator는 keda Namespace에서 동작하도록 구성합니다. Pod 1개 당 처리할 메세지 수는 5건을 목표로 하며, 메세지가 없을 때에는 Pod를 1개로 축소합니다.

- ScaledObject Name / Namespace : order-scaler / skillsmkt
- Min / Max Replica Count : 1 / 5

5. Node Scaling

주문량 증가에 따라 노드 부족으로 인한 문제가 발생하지 않도록 Karpenter를 설치하여 Node가 자동으로 Scale-out/in 될 수 있도록 구성합니다. Karpenter Controller는 kube-system Namespace에서 동작하여야 합니다. NodePool, Class를 아래 설명에 맞게 구성하도록 합니다. 해당 노드에는 App, DaemonSet을 제외한 워크로드가 실행되어선 안되며, 유휴 또는 활용도가 낮은 노드는 60초 후 반환되도록 Consolidation 정책을 구성합니다. 부하 주입 시 Pod가 Max replica로 증설될 때, 기존 노드 1대가 이를 모두 수용할 수 없으므로, Karpenter가 Pending Pod를 감지하여 노드 1대를 추가적으로 프로비저닝 하여야 합니다.

- NodePool / NodeClass Name : skm-app-nodepool / skm-app-nodeclass
- Allowed Instance Types : t3.small, t3.medium

과제 종료 전 모든 부하 주입을 종료하여 Pod 1개, Node 1대만 존재하도록 합니다. 해당 값과 다를 시 채점에 불이익이 있을 수 있습니다. Scale in/out 채점 시 2분 대기함에 유의합니다.

4) Container Logging (ap-northeast-1)

개요

EKS, OpenTelemetry Collector, Loki, Grafana를 사용하여 워크로드의 로그를 수집, 저장, 시각화하는 통합 로깅 시스템을 구성합니다. 자세한 사항은 아래를 참고하여 구성합니다.

1. EKS Cluster

워크로드와 로깅 시스템을 운영할 EKS 클러스터를 구축합니다. 노드 그룹은 Multi-AZ로 배치합니다. 모든 노드의 TimeZone은 한국 표준 시간대로 설정합니다.

- Cluster Name / Version : o11y-cluster / 1.35
- NodeGroup Instance Type : t3.medium
- NodeGroup Min / Desired / Max Size : 2 / 2 / 2

2. Log Producer Application

Python으로 작성된 App app.py를 컨테이너 이미지로 빌드하여 o11y Namespace에 배포합니다. App은 HTTP 요청을 받아 stdout으로 JSON 형식의 로그를 출력하며, 8080 포트에서 동작하도록 구현되었습니다. ALB를 통해 외부에서 호출 가능하도록 구성합니다.

- Deployment Name / Initial Replicas : log-generator / 2
- ALB / TG Name : o11y-app-alb / o11y-app-tg
- API Spec & Log Example

Path	Method	Response / Behavior
/healthz	GET	HTTP 200, {"status": "ok"}
/log?level=L&count=N	GET	HTTP 200, L레벨(info/warn/error)의 로그 N건을 출력

```
{
  "ts": "2026-05-28T14:23:45.123Z",
  "level": "INFO",
  "msg": "log generated",
  "req_id": "a3b2c1d4-7e8f-4901-9234-abcdef012345"
}
```

3. Log Collector

워크로드의 로그를 수집하기 위해 OTel Collector를 DaemonSet 형태로 monitoring Namespace에 배포합니다. /var/log/pods/ 경로에서 컨테이너 로그를 수집하며, Kubernetes Metadata를 enrichment한 뒤 OTLP HTTP 프로토콜로 Loki에 전송합니다. 수집된 로그는 namespace, pod 단위로 Loki에서 필터링 가능해야 합니다.

- DaemonSet Name : o11y-otel
- Receiver / Processor : filelog (/var/log/pods/*/*/*.log) / k8sattributes

4. Log Backend

수집된 로그를 저장하고 LogQL 쿼리를 제공하기 위해 Loki를 Single Binary 모드로 monitoring Namespace에 배포합니다. Chunks와 Index는 Persistent Volume에 저장하며, OTel Collector로부터 OTLP HTTP 프로토콜의 로그를 수신할 수 있도록 OTLP Ingestion Endpoint를 활성화합니다. 다른 Pod에서 ClusterIP Service를 통해 Loki에 접근 가능해야 합니다.

- Release / Service Name : o11y-loki

5. Dashboard

운영자가 수집된 로그를 조회할 수 있도록 Grafana를 monitoring Namespace에 배포합니다. ALB를 통해 외부에서 접근 가능하도록 구성하며, Loki를 Datasource로 등록합니다. 'Log Overview' 대시보드를 구성하여 시간 별 로그 건수(bar chart), 로그 분포(pi chart), 최근 로그(logs)를 시각화합니다. 패널 이름, 배치 등 구성은 아래 이미지와 같게 구성합니다. 범례는 error, log, warn 등 plain text로 출력되어야 하며, 대소문자 구분하지 않습니다.

- Deployment / ALB / TG Name : o11y-grafana / o11y-grafana-alb / o11y-grafana-tg
- Admin ID / PW : skills<선수등번호> / GoodJob!Skills<선수등번호>^^

